Mini Project 11 — Membandingkan dan Membekukan Baseline Machine Learning

DuMini Project 11 menggunakan representasi final MP10 tanpa feature selection ulang. Tugas mahasiswa adalah membandingkan enam keluarga classifier, memilih hyperparameter dengan training GroupKFold, memilih keluarga final menggunakan validation, menyimpan model, dan menjaga test set tetap tidak tersentuh.

Input dari MP10	Output MP11
features/dataset/mp10_features_final.npz	models/mp11_classical_baseline.joblib
config/feature_reduction_config.json	models/mp11_model_parameters.npz
splits dari MP9 yang terbawa dalam NPZ	config/classifier_config.json
project_status.json = ready_for_mp11	results/metrics/mp11_classifier_metrics.json
representasi final beku	3 grafik audit + status ready_for_mp12

Tabel 11.12. Input dan output Mini Project 11.

Tahap	Estimasi
Periksa status dan dependency	2 menit
Simpan/jalankan skrip	3 menit
Tuning enam keluarga model	6–9 menit
Periksa comparison + validation	4–5 menit
Periksa model/config/parameter	3–4 menit
Validasi bahwa test belum digunakan	2 menit

Tabel 11.13. Rencana waktu Mini Project 11.

Prosedur Step by Step

Langkah 1 — Masuk ke root proyek dan periksa status
Pastikan project_status.json bernilai ready_for_mp11. Jangan membuat split baru, jangan mengubah feature_reduction_config.json, dan jangan membuka test label untuk memilih model.

**Langkah 2 — Verifikasi paket **
Pastikan NumPy, Matplotlib, scikit-learn, dan joblib tersedia. Semua data audio mentah sudah diproses pada bab sebelumnya.

Langkah 3 — Simpan skrip Mini Project 11
Buat mini_project_11.py pada root proyek dan salin kode Python pada bagian berikut.

Langkah 4 — Jalankan tuning training-only
Skrip mencoba KNN, logistic regression, linear SVM, decision tree, Random Forest, dan MLP. Setiap family memakai grid kecil dan GroupKFold berdasarkan acquisition block training.

Langkah 5 — Periksa pemilihan model dengan validation
Buka mp11_classifier_comparison.png dan mp11_classifier_metrics.json. Catat chosen_family, validation macro-F1, serta resource proxy. Hasil nyata dapat berbeda dari runtime test buku.

Langkah 6 — Periksa model yang dibekukan
Pastikan classifier_config.json, mp11_classical_baseline.joblib, dan mp11_model_parameters.npz tersedia. Jangan retrain model ini pada validation atau test.

Langkah 7 — Validasi test lock dan handoff
Pastikan test_used=false, test_predictions_generated=false, dan project_status.json berubah menjadi ready_for_mp12.

Kode Python Mini Project 11

```
from pathlib import Path
import json, pickle, warnings
import numpy as np
import matplotlib.pyplot as plt
from sklearn.base import clone
from sklearn.model_selection import GroupKFold
from sklearn.metrics import f1_score, accuracy_score, confusion_matrix
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
import joblib

warnings.filterwarnings('ignore')
ROOT = Path('.')
SEED = 2026
VAL_TOL = 0.03


def save_json(path, obj):
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(obj, indent=2), encoding='utf-8')


def as_py(v):
    if isinstance(v, (np.integer,)): return int(v)
    if isinstance(v, (np.floating,)): return float(v)
    if isinstance(v, np.ndarray): return v.tolist()
    if isinstance(v, tuple): return list(v)
    return v


def macro_f1(y_true, y_pred):
    return f1_score(y_true, y_pred, average='macro', zero_division=0)


def cv_score(estimator, X, y, groups):
    unique_groups = np.unique(groups)
    n_splits = min(4, len(unique_groups))
    if n_splits < 2:
        raise RuntimeError('Training set membutuhkan minimal dua acquisition group.')
    cv = GroupKFold(n_splits=n_splits)
    scores = []
    for tr, va in cv.split(X, y, groups):
        model = clone(estimator)
        model.fit(X[tr], y[tr])
        pred = model.predict(X[va])
        scores.append(macro_f1(y[va], pred))
    return float(np.mean(scores)), float(np.std(scores)), [float(s) for s in scores]


def model_stats(name, model, X_train):
    # Perkiraan operasi primitif per sampel, dipakai hanya sebagai pembanding relatif.
    d = X_train.shape[1]
    if name == 'knn':
        ops = int(X_train.shape[0] * d * 3)
        params = int(X_train.size + X_train.shape[0])
    elif name in ('logreg', 'linear_svm'):
        coef = np.asarray(model.coef_)
        ops = int(coef.size * 2)
        params = int(coef.size + np.asarray(model.intercept_).size)
    elif name == 'tree':
        ops = int(model.get_depth())
        params = int(model.tree_.node_count)
    elif name == 'rf':
        ops = int(sum(est.get_depth() for est in model.estimators_))
        params = int(sum(est.tree_.node_count for est in model.estimators_))
    elif name == 'mlp':
        params = int(sum(w.size for w in model.coefs_) + sum(b.size for b in model.intercepts_))
        ops = int(2 * sum(w.size for w in model.coefs_))
    else:
        ops = params = 0
    size = len(pickle.dumps(model, protocol=pickle.HIGHEST_PROTOCOL))
    return {'estimated_ops_per_sample': ops, 'parameter_count_proxy': params, 'serialized_bytes': int(size)}


status_path = ROOT / 'project_status.json'
if not status_path.exists():
    raise FileNotFoundError('project_status.json tidak ditemukan. Jalankan Mini Project sebelumnya.')
status = json.loads(status_path.read_text(encoding='utf-8'))
if status.get('status') != 'ready_for_mp11':
    raise RuntimeError(f"Status harus ready_for_mp11, ditemukan {status.get('status')!r}")
```

```
src = ROOT / 'features/dataset/mp10_features_final.npz'
if not src.exists():
    raise FileNotFoundError(src)
D = np.load(src, allow_pickle=True)
X = D['X'].astype(np.float64)
y = D['y'].astype(str)
groups = D['groups'].astype(str)
train_mask = D['train_mask'].astype(bool)
val_mask = D['val_mask'].astype(bool)
test_mask = D['test_mask'].astype(bool)
sample_ids = D['sample_ids'].astype(str)
if not np.isfinite(X).all():
    raise RuntimeError('Dataset MP10 mengandung NaN/Inf.')
if train_mask.sum() < 8 or val_mask.sum() < 4 or test_mask.sum() < 4:
    raise RuntimeError('Split MP9/MP10 tidak sesuai running project.')

Xtr, ytr, gtr = X[train_mask], y[train_mask], groups[train_mask]
Xv, yv = X[val_mask], y[val_mask]
classes = np.unique(y)

families = {
    'knn': [
        {'n_neighbors': 1, 'weights': 'uniform'},
        {'n_neighbors': 3, 'weights': 'uniform'},
        {'n_neighbors': 3, 'weights': 'distance'},
    ],
    'logreg': [
        {'C': 0.1}, {'C': 1.0}, {'C': 10.0},
    ],
    'linear_svm': [
        {'C': 0.1}, {'C': 1.0}, {'C': 10.0},
    ],
    'tree': [
        {'max_depth': 2}, {'max_depth': 3}, {'max_depth': 4},
    ],
    'rf': [
        {'n_estimators': 30, 'max_depth': 3},
        {'n_estimators': 50, 'max_depth': 4},
    ],
    'mlp': [
        {'hidden_layer_sizes': (8,), 'alpha': 1e-3},
        {'hidden_layer_sizes': (16,), 'alpha': 1e-3},
    ],
}


def build(name, p):
    if name == 'knn':
        return KNeighborsClassifier(**p)
    if name == 'logreg':
        return LogisticRegression(max_iter=2000, solver='lbfgs', random_state=SEED, **p)
    if name == 'linear_svm':
        return LinearSVC(dual='auto', random_state=SEED, **p)
    if name == 'tree':
        return DecisionTreeClassifier(random_state=SEED, min_samples_leaf=1, **p)
    if name == 'rf':
        return RandomForestClassifier(random_state=SEED, n_jobs=1, **p)
    if name == 'mlp':
        return MLPClassifier(max_iter=2500, activation='relu', solver='lbfgs', random_state=SEED, **p)
    raise KeyError(name)

# 1. Tune hyperparameter per family dengan training GroupKFold saja.
family_results = {}
models = {}
for family, grid in families.items():
    trials = []
    for hp in grid:
        est = build(family, hp)
        mean_f1, std_f1, fold_scores = cv_score(est, Xtr, ytr, gtr)
        trials.append({'params': {k: as_py(v) for k,v in hp.items()},
                       'cv_macro_f1_mean': mean_f1,
                       'cv_macro_f1_std': std_f1,
                       'fold_macro_f1': fold_scores})
    best = sorted(trials, key=lambda r: (-r['cv_macro_f1_mean'], r['cv_macro_f1_std']))[0]
    model = build(family, best['params'])
    # tuple MLP disimpan JSON sebagai list; konversi kembali saat perlu.
    if family == 'mlp' and isinstance(best['params'].get('hidden_layer_sizes'), list):
        hp2 = dict(best['params']); hp2['hidden_layer_sizes'] = tuple(hp2['hidden_layer_sizes'])
        model = build(family, hp2)
    model.fit(Xtr, ytr)
    pred_val = model.predict(Xv)
    val_f1 = macro_f1(yv, pred_val)
    val_acc = accuracy_score(yv, pred_val)
    stats = model_stats(family, model, Xtr)
    family_results[family] = {
        'best_params': best['params'],
```

```
        'cv_macro_f1_mean': best['cv_macro_f1_mean'],
        'cv_macro_f1_std': best['cv_macro_f1_std'],
        'validation_macro_f1': float(val_f1),
        'validation_accuracy': float(val_acc),
        'resource_proxy': stats,
        'all_trials': trials,
        'validation_predictions': pred_val.tolist(),
    }
    models[family] = model

# 2. Pilih keluarga final dengan validation, lalu biaya sebagai tie-break.
best_val = max(r['validation_macro_f1'] for r in family_results.values())
shortlist = [n for n,r in family_results.items() if r['validation_macro_f1'] >= best_val - VAL_TOL]
# Bias kecil ke model yang CV-nya lebih stabil, lalu operasi dan ukuran lebih kecil.
chosen = sorted(shortlist, key=lambda n: (
    -family_results[n]['validation_macro_f1'],
    -family_results[n]['cv_macro_f1_mean'],
    family_results[n]['resource_proxy']['estimated_ops_per_sample'],
    family_results[n]['resource_proxy']['serialized_bytes'],
))[0]
model = models[chosen]
val_pred = model.predict(Xv)
cm = confusion_matrix(yv, val_pred, labels=classes)

# 3. Simpan model dan artefak. Test set sengaja tidak diprediksi.
model_dir = ROOT / 'models'; model_dir.mkdir(parents=True, exist_ok=True)
result_dir = ROOT / 'results/metrics'; result_dir.mkdir(parents=True, exist_ok=True)
fig_dir = ROOT / 'results/figures'; fig_dir.mkdir(parents=True, exist_ok=True)
config_dir = ROOT / 'config'; config_dir.mkdir(parents=True, exist_ok=True)

model_path = model_dir / 'mp11_classical_baseline.joblib'
joblib.dump(model, model_path)

# Parameter numerik ringkas untuk porting/audit.
param_path = model_dir / 'mp11_model_parameters.npz'
param_payload = {'classes': np.array(classes, dtype=object), 'model_family': np.array([chosen], dtype=object)}
if chosen in ('logreg','linear_svm'):
    param_payload['coef'] = np.asarray(model.coef_, dtype=np.float32)
    param_payload['intercept'] = np.asarray(model.intercept_, dtype=np.float32)
elif chosen == 'tree':
    param_payload.update({
        'children_left': model.tree_.children_left.astype(np.int32),
        'children_right': model.tree_.children_right.astype(np.int32),
        'feature': model.tree_.feature.astype(np.int32),
        'threshold': model.tree_.threshold.astype(np.float32),
        'value': model.tree_.value.astype(np.float32),
    })
elif chosen == 'knn':
    param_payload['train_X'] = model._fit_X.astype(np.float32)
    param_payload['train_y_encoded'] = np.asarray(model._y, dtype=np.int32)
elif chosen == 'mlp':
    for i,w in enumerate(model.coefs_): param_payload[f'W{i}'] = np.asarray(w, dtype=np.float32)
    for i,b in enumerate(model.intercepts_): param_payload[f'b{i}'] = np.asarray(b, dtype=np.float32)
elif chosen == 'rf':
    param_payload['n_estimators'] = np.array([len(model.estimators_)], dtype=np.int32)
np.savez_compressed(param_path, **param_payload)

if chosen == 'tree':
    (model_dir/'mp11_tree_rules.txt').write_text(export_text(model), encoding='utf-8')

config = {
    'source_dataset': 'features/dataset/mp10_features_final.npz',
    'feature_representation_frozen_in_mp10': True,
    'hyperparameter_fit_scope': 'training_only_groupkfold',
    'family_selection_scope': 'validation_only_after_family_tuning',
    'test_used_for_tuning': False,
    'test_used_for_family_selection': False,
    'test_predictions_generated': False,
    'seed': SEED,
    'validation_f1_tolerance': VAL_TOL,
    'chosen_family': chosen,
    'chosen_params': family_results[chosen]['best_params'],
    'classes': classes.tolist(),
    'input_dim': int(X.shape[1]),
    'model_file': str(model_path),
    'parameter_file': str(param_path),
}
save_json(config_dir/'classifier_config.json', config)

report = {
    'n_samples_total': int(len(y)),
    'n_train': int(train_mask.sum()),
    'n_validation': int(val_mask.sum()),
    'n_test_reserved_untouched': int(test_mask.sum()),
    'input_dim': int(X.shape[1]),
    'classes': classes.tolist(),
    'family_results': family_results,
```

```
    'best_validation_macro_f1': float(best_val),
    'shortlist_within_tolerance': shortlist,
    'chosen_family': chosen,
    'chosen_validation_macro_f1': float(family_results[chosen]['validation_macro_f1']),
    'chosen_validation_accuracy': float(family_results[chosen]['validation_accuracy']),
    'validation_confusion_matrix': cm.tolist(),
    'test_used': False,
}
save_json(result_dir/'mp11_classifier_metrics.json', report)

# 4. Figures.
names = list(family_results.keys())
cv_vals = [family_results[n]['cv_macro_f1_mean'] for n in names]
val_vals = [family_results[n]['validation_macro_f1'] for n in names]
xpos = np.arange(len(names))
plt.figure(figsize=(8.4,4.8))
plt.bar(xpos-0.18, cv_vals, width=0.36, label='Training GroupKFold')
plt.bar(xpos+0.18, val_vals, width=0.36, label='Validation block')
plt.xticks(xpos, names, rotation=20)
plt.ylabel('Macro-F1'); plt.ylim(0,1.05); plt.title('MP11 - Perbandingan classifier')
plt.legend(); plt.grid(axis='y', alpha=.2); plt.tight_layout()
plt.savefig(fig_dir/'mp11_classifier_comparison.png', dpi=170); plt.close()

sizes = [family_results[n]['resource_proxy']['serialized_bytes'] for n in names]
ops = [family_results[n]['resource_proxy']['estimated_ops_per_sample'] for n in names]
plt.figure(figsize=(7.4,5.0))
plt.scatter(ops, sizes, s=80)
for n,a,b in zip(names,ops,sizes): plt.annotate(n,(a,b),xytext=(4,4),textcoords='offset points',fontsize=8)
plt.xlabel('Estimated primitive operations / sample (proxy)')
plt.ylabel('Serialized model bytes (proxy)')
plt.title('MP11 - Trade-off sumber daya classifier')
plt.grid(alpha=.2); plt.tight_layout()
plt.savefig(fig_dir/'mp11_resource_tradeoff.png', dpi=170); plt.close()

plt.figure(figsize=(5.4,4.7))
plt.imshow(cm, cmap='Blues')
plt.colorbar(label='Jumlah sampel')
plt.xticks(range(len(classes)), classes, rotation=30); plt.yticks(range(len(classes)), classes)
plt.xlabel('Prediksi'); plt.ylabel('Label aktual'); plt.title(f'Validation confusion matrix - {chosen}')
for i in range(len(classes)):
    for j in range(len(classes)):
        plt.text(j,i,str(cm[i,j]),ha='center',va='center')
plt.tight_layout(); plt.savefig(fig_dir/'mp11_validation_confusion_matrix.png',dpi=170); plt.close()

# 5. Update status for MP12.
status['status'] = 'ready_for_mp12'
status['last_completed_mini_project'] = 11
status['mp11'] = {
    'chosen_family': chosen,
    'classifier_config': 'config/classifier_config.json',
    'model_file': str(model_path),
    'parameter_file': str(param_path),
    'metrics': 'results/metrics/mp11_classifier_metrics.json',
    'test_used': False,
}
save_json(status_path, status)

print('Mini Project 11 selesai.')
print('Chosen family:', chosen)
print('Validation macro-F1:', f"{family_results[chosen]['validation_macro_f1']:.3f}")
print('Test used:', False)
print('Status:', status['status'])
```

Artefak yang Wajib Dibawa ke Mini Project 12

- `config/classifier_config.json` — model family dan hyperparameter baseline klasik yang sudah dibekukan.
- `models/mp11_classical_baseline.joblib` — baseline klasik yang akan disimpan sampai evaluasi Bab 13.
- `models/mp11_model_parameters.npz` — parameter numerik untuk audit dan persiapan deployment.
- `results/metrics/mp11_classifier_metrics.json` — hasil training-CV, validation, dan resource proxy.
- `features/dataset/mp10_features_final.npz` — tetap menjadi input baseline klasik; tidak diubah.
- `project_status.json` dengan status `ready_for_mp12`.

Mini Project 12 tidak mengganti baseline klasik. Bab 12 membangun jalur deep learning/TinyML dari representasi time-frequency yang sudah disimpan sejak MP8–MP9. Kedua jalur baru dibandingkan secara final pada Bab 13 menggunakan test set yang masih terkunci.

Referensi Bab

1. Bishop, C. M. Pattern Recognition and Machine Learning. Springer.

2. Hastie, T., Tibshirani, R., dan Friedman, J. The Elements of Statistical Learning. Springer.

3. Cover, T., dan Hart, P. Nearest Neighbor Pattern Classification. IEEE Transactions on Information Theory.

4. Cortes, C., dan Vapnik, V. Support-Vector Networks. Machine Learning.

5. Breiman, L. Random Forests. Machine Learning.

6. Quinlan, J. R. C4.5: Programs for Machine Learning. Morgan Kaufmann.

7. Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly.

8. Scikit-learn documentation: nearest neighbors, linear models, SVM, trees, ensembles, neural networks, and model selection.